# Secure Development and Deployment – COM6003 - 2306422

# Business Requirements Document

HospiManagementApp is a secure Android-based hospital management application commissioned to streamline administrative and clinical operations. The system integrates patient registration, appointment scheduling, electronic health records (EHR), and security into a user-friendly mobile platform. The overarching goal is to implement hospital operations in an app by providing a secure, efficient digital platform for managing patients, healthcare professionals, and appointments, while ensuring full compliance with healthcare data protection regulations including UK GDPR and the NHS Data Security and Protection Toolkit (DSPT). This document outlines the key stakeholders, core system epics, user stories, acceptance criteria, evil user stories and development roadmap across four incremental sprints.

**Business Goals:**

- Digitise patient registration.
- Provide a secure, role-based platform for managing clinic appointments.
- Enable authorised staff to access and update patient records.
- Ensure all patient data is stored securely and in compliance with regulations.

**Stakeholders:**

- Hospital Administrator: manages staff accounts and system access.
- Clinician: views and updates patients EHR's.
- Receptionist: books and manages appointments.
- Patient: personal information is stored in database.
- Regulators: ensure app is compliant.

**Success Criteria:**

- Sensitive patient data is validated and hosted in an encrypted Room database.
- Staff authentication uses hashed credentials and role-based access control throughout.
- Appointments can be viewed, booked, and managed with conflict detection and biometric authentication.
- Patient EHRs are accessible, editable by authorised roles, and protected by field-level encryption.
- The application demonstrates compliance with UK GDPR and NHS DSPT requirements.
- Testing validates security-critical features including NHS validation, hashing, and input sanitisation.

**Epics:**

The core epics are provided here, with associated user stories and acceptance criteria. Behaviour Driven Development (BDD) methods have been used to develop these features, with a "Given, Then, When" format for the acceptance criteria.

| Epic | User Story | Acceptance Criteria |
|---|---|---|
| Patient Management | As a receptionist, I want to register a patient with their NHS number so that accurate records are created. | **Given** a receptionist enters patient details including an NHS number,<br>**When** the Save Patient button is tapped,<br>**Then:**<br>- NHS number validated.<br>- No duplicate NHS numbers across patients.<br>- No missing NHS numbers for patients.<br>- Records stored in database.<br>- Error handling for missing or incorrectly formatted input. |
| Admin Authentication | As an administrator, I want to log in with my email and PIN so that I can access staff management features. | **Given** an administrator enters their email and PIN<br>**When** the Login button is tapped,<br>**Then:**<br>- Login fails with invalid credentials.<br>- Account locked after 3 consecutive failed attempts.<br>- PIN verified against stored SHA-256 hash, nothing in plaintext.<br>- No duplicate emails across staff.<br>- Role Based Access. |

| | | |
|---|---|---|
| Appointment Scheduling | As a receptionist, I want to book appointments without conflicts so that no clinician is double-booked. | **Given** a receptionist selects an appointment slot for a clinician<br>**When** the Confirm button is tapped<br>**Then:**<br>- Appointments displayed and filtered by clinic<br>- Conflict checks before appointment confirmation.<br>- Booking restricted to admin and reception roles.<br>- Success and error feedback provided via toast messages. |
| Patient Records (EHR) | As a clinician, I want to view and update a patient's EHR so that I have accurate information during consultation. | **Given** a clinician scans a patient wristband or enters an NHS number<br>**When** the patient summary screen opens<br>**Then:**<br>- Conditions, allergies, and medications displayed after decryption.<br>- EHR barcode scanner to get patient information.<br>- Edit access restricted to Clinician and Admin.<br>- Confidential patient information decrypted for display only.<br>- Vital signs recorded with offline queue<br>- No sensitive information to appear in system logs at any point |
| Security and Compliance | As an administrator, I want patient data encrypted so that a device breach does not expose confidential patient information. | **Given** a clinical record is saved to the database<br>**When** the database file is inspected directly<br>**Then:**<br>- AES-256 applied to relevant fields.<br>- Encryption key stored in Android Keystore, not in app memory<br>- SQLCipher encrypted database files.<br>- Tampering detection. |
| Development Requirements | As a developer, I want the application to be modular so that it allows ease of integration with other functions and code. | **Given** a developer wants to understand the system structure,<br>**When** the developer accesses the codebase,<br>**Then:**<br>- Entity class contain sufficient information about variables<br>- Comments explain the purpose of functions<br>- Package structure follows clean architecture |

**Evil User Stories:**

Evil user stories anticipate how a malicious actor might attempt to exploit the system. By identifying these threats during the planning phase, security controls can be embedded from the ground up rather than added retrospectively. Each story here is paired with a mitigation strategy.

| Evil User Story | Threat | Mitigation |
|---|---|---|
| As an attacker, I want to inject SQL via the NHS number field to extract the patient database. | SQL Injection | Room parameterised queries prevent injection at the data layer. sanitiseInput() strips SQL keywords as a secondary defence. |
| As an attacker, I want to brute-force the admin PIN to gain elevated access | Credential attack | ThreatMonitor locks the account after 3 failures. PINs are stored as SHA-256 hashes. |
| As an attacker, I want to extract the database file and read patient records. | Data breach | SQLCipher AES-256 encrypts the database file. SecurityAgent applies encryption to all sensitive data |
| As an attacker, I want to modify a stored EHR record to corrupt patient data | Data tampering | Authentication tags detect ciphertext modification. SecurityAgent logs a TAMPERING ALERT on decryption failure. |

**Methodology:**

We will develop the app using an agile methodology. This allows for the development to be broken down into distinct stages with room for reflection and alterations. Iterative delivery allows for security to be built up layer by layer way as the app grows across sprints. Each lab/sprint includes a small number of distinct aspects or features. This modularity allows for important features to be built well and not rushed. Solid foundational work in each sprint reduces risk of having to rebuild a feature or start a slice again. Lab 2 cannot be built without Lab 1's Room database and relies on it being fit for purpose. Similarly, Lab 4's encryption depends on Lab 3's EHR structure. Agile also allows for great scalability, new features can be added with minimal restructuring as it is already modular. Short sprint cycles allow issues identified during dynamic testing to be addressed before the next sprint begins.

**Timeline:**

**Lab 1 - Foundation and Patient Registration**
*Epics:* Patient Management, Admin Authentication
*Key Implementations:* Patient registration, NHS number validation, RBAC, SHA-256 PIN hashing, Room DB
*Tools/Libraries:* Room 2.6.1, SharedPreferences, Java MessageDigest

**Lab 2 - Appointments and Core Security**
*Epics:* Appointment Management**,** Network layer, Authentication
*Key Implementations:* Appointment list, conflict detection, biometric auth, Retrofit mock API, RetryInterceptor
*Tools/Libraries:* Retrofit 2.11, OkHttp 4.12, BiometricPrompt

**Lab 3 - Clinical Records and Advanced Features**
*Epics:* Clinical Records (EHR), Barcode Scanning, Pagination
*Key Implementations:* EHR CRUD, vital signs, barcode scanning, WorkManager offline sync, Pagination
*Tools/Libraries:* Room Paging, WorkManager 2.9.1, ZXing 4.3.0, Executors

**Lab 4 - Enhanced Comprehensive Security**
*Epics:* Security and Compliance
*Key Implementations:* AES-256/GCM field encryption, Android Keystore, SQLCipher, ThreatMonitor, comprehensive unit testing
*Tools/Libraries:* SQLCipher 4.5.4, Android Keystore API, JUnit 4, R8

**Security Principles:**
Each stage of development is carried out with the following security principles in mind.

*Confidentiality*: Patient data is accessible only to authenticated users with the appropriate role. AES-256/GCM field-level encryption and SQLCipher database encryption ensure sensitive patient information cannot be read from storage or backups without the Android Keystore key. RBAC gates are applied to prevent unauthorised access to clinical screens.

*Integrity:* GCM authentication tags automatically detect any modification to encrypted records. NHS Mod 11 validation and sanitiseInput() prevent corrupt or malicious data entering the system. SHA-256 PIN hashing ensures stored credentials cannot be modified without the change being detectable at login.

*Availability*: WorkManager ensures vital signs recorded offline are queued with

synced=false and synchronised automatically when network connectivity is restored. RetryInterceptor handles transient API failures gracefully. fallbackToDestructiveMigration() prevents database schema conflicts blocking application startup across lab upgrades.

*Non-repudiation:* The updatedAt and recordedBy fields on EHR records provide an immutable audit trail of who modified clinical data and when. ThreatMonitor logs security events with counts and timestamps. SessionManager records the authenticated user's role and email, associating all privileged actions with a specific account.